

CITYPLAY : Répartition des Tâches

Projet : Jeu de découverte urbaine (Laravel / Vue.js / Geofencing)

Objectif : Répartition optimale pour une équipe de 3 développeurs avec une réduction maximale des dépendances inter-modules.

1. Architecture de Découpage

Développeur	Domaine Principal	Objectif Métier
Développeur 1	Back-office Admin & Contenu	Gestion du catalogue (Villes, Lieux, Énigmes, Médias).
Développeur 2	Auth, Invitations & Sessions	Gestion des accès, cycles de vie des sessions et lobby.
Développeur 3	Gameplay, GPS & Scoring	Expérience temps réel, carte interactive et notifications.

2. Propriété des Données (Tables)

Chaque table a un responsable principal chargé de sa structure et de sa logique d'écriture.

Table	Responsable	Usage externe
users	Dev 2	Lecture Admin (Dev 1), Scores (Dev 3)
cities / places / riddles / hints	Dev 1	Sessions (Dev 2), Validation (Dev 3)
invitations / game_sessions	Dev 2	Validation & Carte (Dev 3)
scores / achievements / notifications	Dev 3	Profil/Résumé (Dev 2)
hint_usages / riddle_locks / position_logs	Dev 3	Suivi gameplay spécifique

3. Missions par Développeur

Développeur 1 : Le "Master du Contenu"

Responsable de l'usine à gaz administrative permettant de créer les parcours.

- **CRUD Villes** : Configuration GPS centre-ville, bannières, descriptions.
- **Lieux & Parcours** : Gestion de l'ordre (order_index) et rayons de validation.
- **Énigmes & Indices** : Types de réponses (GPS/Texte), difficultés, malus des indices.
- **Service Média** : Logique d'upload et redimensionnement d'images.

Structure : app/Services/Content/ | resources/js/Pages/Admin/

Développeur 2 : Le "Gardien des Sessions"

Responsable de l'entrée dans l'application et de la synchronisation de l'équipe.

- **Authentification** : Roles (Admin/Player) via Sanctum/Spatie.
- **Système d'Invitations** : Génération de Tokens uniques et QR Codes.
- **Lobby & Cycle de vie** : État de la session (Pending, Active, Paused, Completed).
- **Gestion d'Équipe** : Suivi des joueurs connectés (GamePlayerService).

Structure : app/Services/Invitation/ | resources/js/Pages/Game/Lobby.vue

Développeur 3 : L' "Ingénieur Gameplay"

Responsable de ce que le joueur voit et ressent sur le terrain.

- **Carte Interactive** : Leaflet.js, position temps réel et markers.
- **Moteur de Validation** : Formule Haversine (GPS), Tolérance Levenshtein (Texte).
- **Scoring & Badges** : Calcul automatique des points (bonus temps/indices).
- **Temps Réel & Notifications** : Intégration Reverb/Pusher pour les événements.

Structure : app/Services/Game/ | resources/js/Pages/Game/Map.vue

4. Contrats et Communication

Pour limiter les frictions, les développeurs communiquent via des **Services de Lecture** et des **Événements**.

Flux recommandé :

- **Dev 1 → Tous** : Fournit `CatalogReadService` pour extraire les données froides.
- **Dev 2 → Dev 3** : Fournit `SessionReadService` pour l'état actuel de la partie.
- **Dev 3 → Dev 2** : Déclenche des événements (ex: `RiddleResolved`) pour mettre à jour l'état de la session.

5. Planning Sprints (5 phases)

Sprint	Focus
1. Base	Migrations, Modèles et Setup Reverb.
2. Modules	CRUD Villes/Lieux, Auth/QR Code, Carte Leaflet.
3. Fonctionnel	CRUD Énigmes, Lobby, Moteur de Validation GPS/Texte.
4. Émotion	Scores, Badges, Notifications, Temps Réel.
5. Polish	Anti-triche, Tests Feature, Optimisations UX.

Règle d'Or : "Un seul propriétaire par table."

On ne modifie pas les données d'un collègue sans passer par son Service dédié.